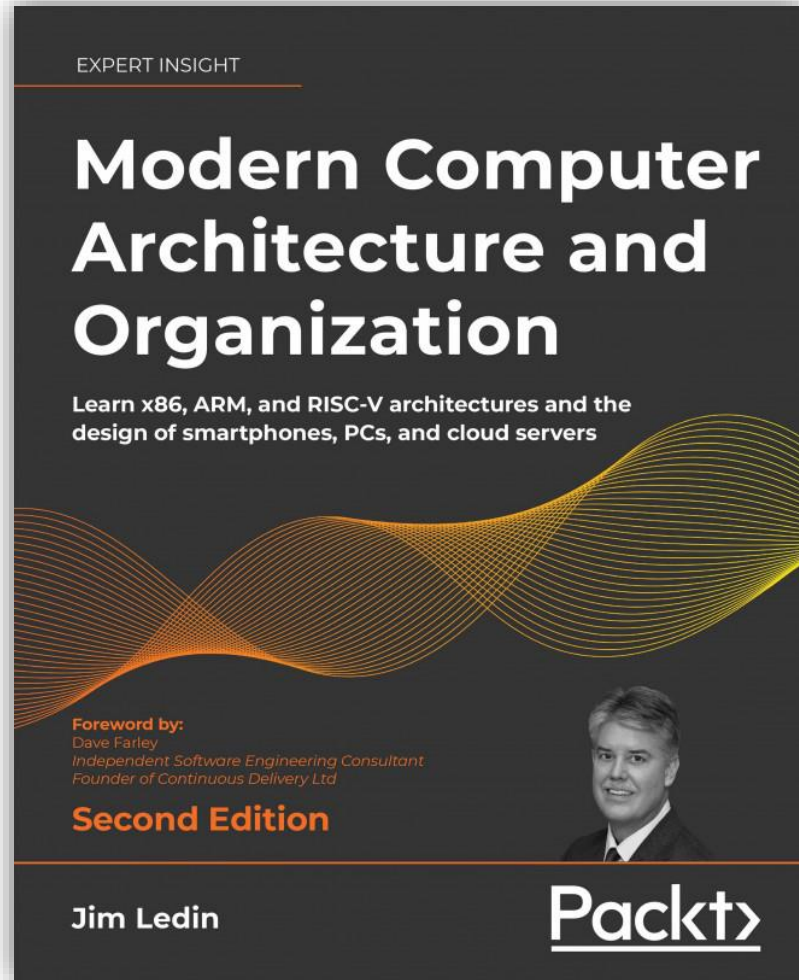


Computer Architecture

Chapter 7 – Processor and Memory Architectures



VIVES University of Applied Sciences Bachelor in electronics-ICT



— Computer Architecture

Chapter 7: Processor and Memory Architectures

Contents

1. **The von Neumann, Harvard and modified Harvard architectures**
2. Physical and virtual memory
3. Paged virtual memory
4. Memory management unit

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- **The von Neumann architecture**

- The von Neumann architecture was introduced by **John von Neumann** in **1945**.
- This processor configuration consists of a **control unit**, an **arithmetic logic unit**, a **register set**, and a **memory region** containing program instructions and data.
- The key feature distinguishing the von Neumann architecture from the Harvard architecture is the use of a **single area of memory** for **program instructions** and the **data** acted upon by those instructions.
- It is conceptually straightforward for programmers, and relatively easier for circuit designers, to locate all of the code and data a program requires in a single memory region.
- **Figure 7.1** shows the elements of the von Neumann architecture.
- Although the single-memory architectural approach simplified the design and construction of early generations of processors and computers, the use of shared program and data memory has presented some significant challenges related to system performance and, in recent years, **security**.

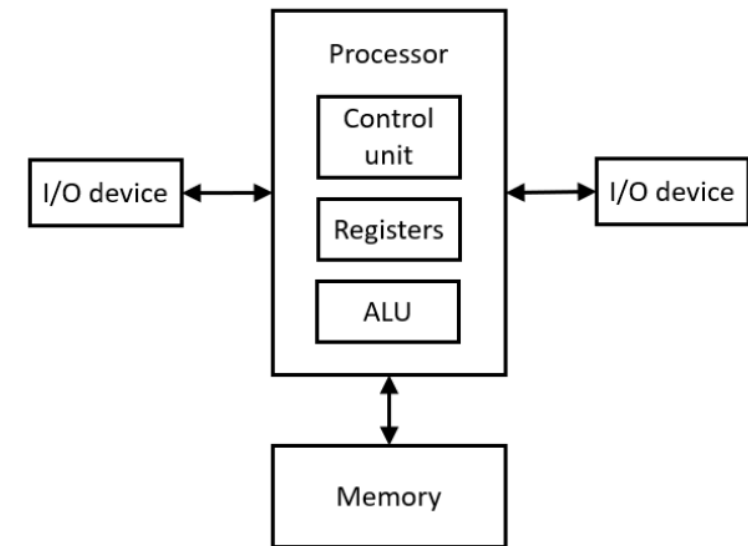


Figure 7.1: The von Neumann architecture

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures: The von Neumann architecture

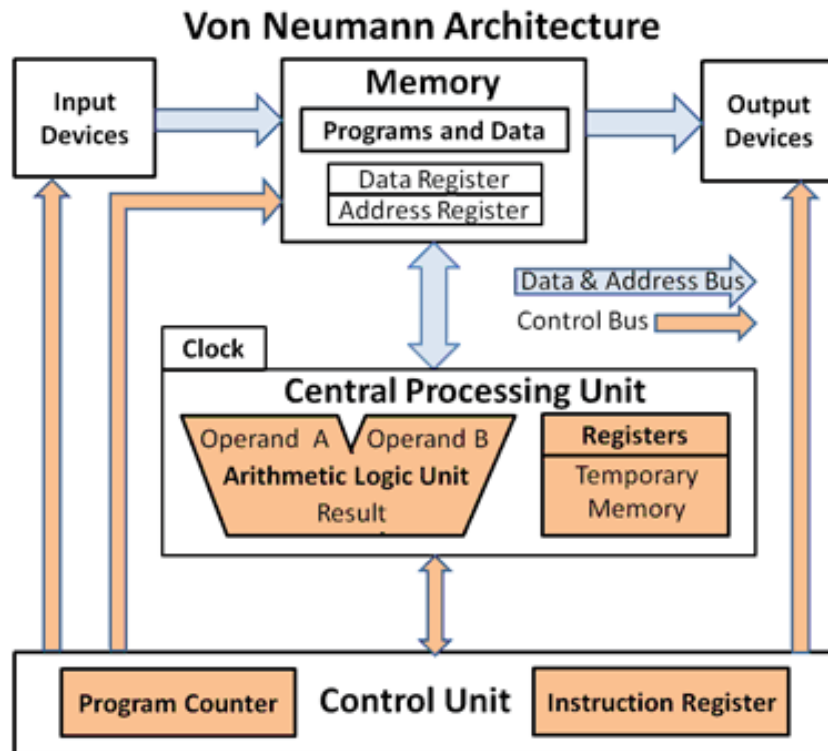


Figure 7.2: The von Neumann architecture

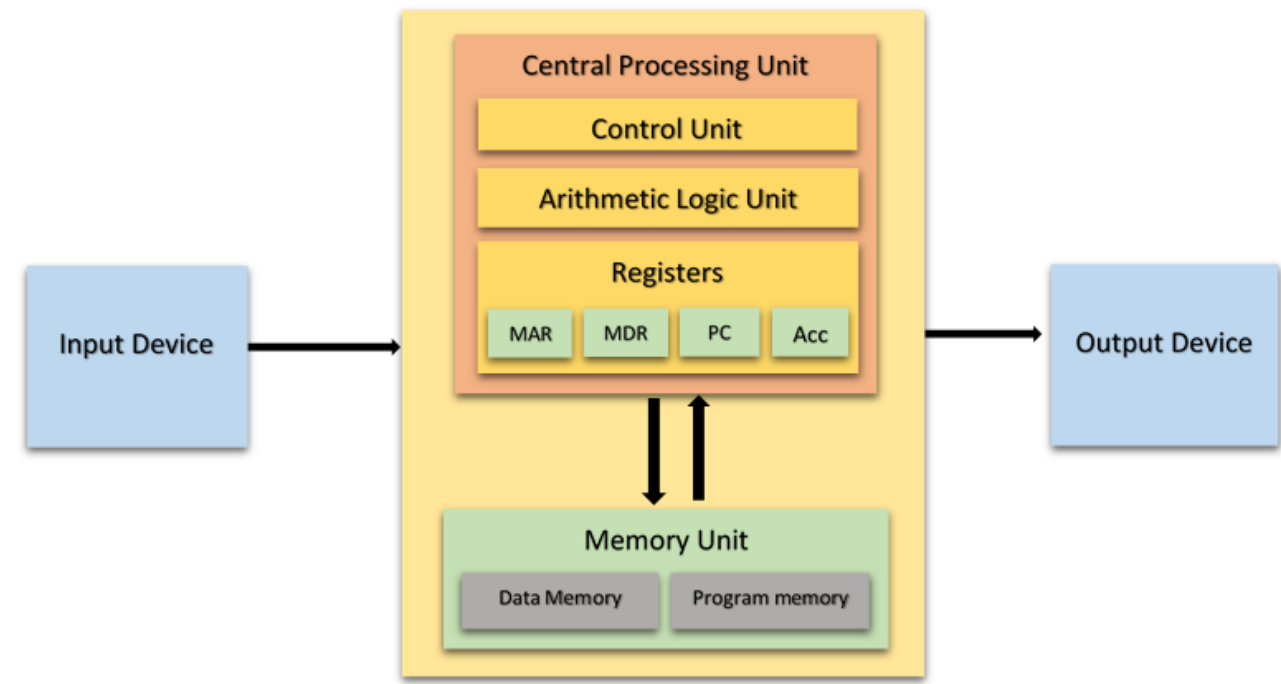


Figure 7.3: The von Neumann architecture

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- Some of the more significant issues were as follows:
 - **The von Neumann bottleneck**: Using a single interface between the processor and the main memory for instruction and data access frequently requires multiple memory cycles to retrieve a single instruction and to access the data it requires.
 - In the case of an immediate value stored next to its instruction opcode, there might be little or no bottleneck penalty because, at least in some cases, the immediate value is loaded along with the opcode in a single memory access.
 - Most programs, however, spend much of their time working with **data stored in memory allocated separately from the program instructions**.
 - In this situation, **multiple memory access operations** are required to retrieve the opcode and any required data items.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- Some of the more significant issues were as follows:
 - The use of **cache** memories for program instructions and data, discussed in detail in *Chapter 8, Performance-Enhancing Techniques*, can significantly mitigate this limitation.
 - However, when working with code sequences and data objects *that exceed the size of cache memory*, the benefit of caching is reduced, possibly by a substantial amount.
 - There is no avoiding the fact that placing code and data in the same memory region with a shared communication path to the processor will, at times, act as a **limitation** on **system performance**.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- Some of the more significant issues were as follows:
 - **The von Neumann security considerations:**
 - The use of a single memory area for code and data opens possibilities for creative programmers to store sequences of instructions in memory as "data," and direct the processor to **execute** those instructions.
 - **Programs that write code into memory and execute it** are implementing **self-modifying code**.
 - Besides being difficult to troubleshoot and debug (because many software debugging tools expect the program in memory to contain the instructions that were originally compiled into it), this capability has been exploited for years by **hackers** with more sinister motives.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- Some of the more significant issues were as follows:
 - **Buffer overflow**: is a distressingly common flaw in widely used software tools such as operating systems, web servers, and databases.
 - **Buffer overflow** occurs when a program requests input and stores that input in a **fixed-length data buffer**. If the code is not careful to check the **length** of the input provided by the user, it is possible for the user to enter an input sequence **longer** than the available storage space.
 - **When this happens, the additional data overwrites memory intended for other purposes.**
 - If the buffer being overwritten is stored on the **program's stack**, it is possible for a creative user to provide a lengthy input sequence that overwrites the **return address** of the currently executing function, which happens to be stored on the same stack.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- Some of the more significant issues were as follows:
 - By carefully crafting the contents of the input data sequence, the attacker can seize control of the executing application and direct it to execute any desired sequence of instructions.
 - *To do this, the hacker must prepare an input sequence that **overflows** the **input buffer**, overwrites the function's **return address** with a different, carefully chosen address, and writes a sequence of instructions into memory that begins execution at this address.*
 - *The sequence of instructions inserted by the attacker begins execution when the function that originally requested user input returns, transferring control to the hacker's code.*
 - This allows the hacker to "own" the computer.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- Some of the more significant issues were as follows:
 - **Buffer overflow:**
 - Various attempts to resolve the buffer overflow problem have occupied an enormous amount of computer security researchers' time over the years since the first widespread occurrence of this type of attack in 1988.
 - Processor vendors and operating system developers have implemented a variety of features to combat buffer overflow attacks, such as **data execution prevention (DEP)** and **address space layout randomization (ASLR)**.
 - While these fixes have been effective to some degree, the fundamental processor feature that enables this type of exploitation is the use of the **same memory region for program instructions and data** in the von Neumann architecture.
 - **Data Execution Prevention Windows (DEP):**
 - <https://docs.microsoft.com/en-us/windows/win32/memory/data-execution-prevention>
 - **Address space layout randomization (ASLR):**
 - https://en.wikipedia.org/wiki/Address_space_layout_randomization

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- **The Harvard architecture**

- The Harvard architecture was originally implemented in the **Harvard Mark I computer** in **1944**.
- A strict Harvard architecture uses one address space and memory bus for **program instructions** and a separate address space and memory bus for **data**.
- This configuration has the immediate benefit of enabling **simultaneous access to instructions and data**, thereby implementing a form of **parallelism**.
- Of course, this enhancement comes at the expense of essentially duplicating the number of address lines, data lines, and control signals that must be implemented by the processor to access both of the memory regions.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- **The Harvard architecture**
- The following diagram shows the layout of a processor implementing the Harvard architecture:

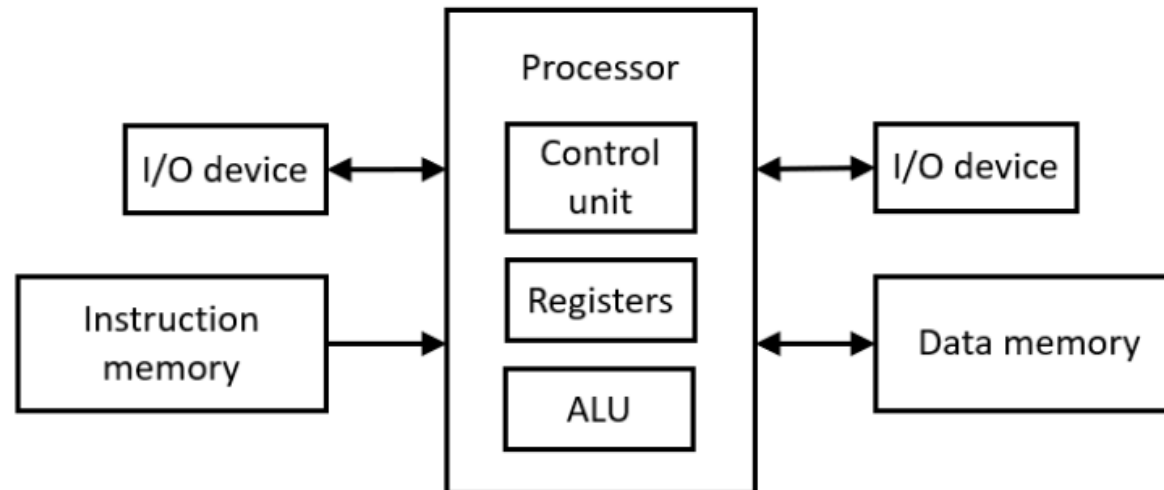


Figure 7.4: Harvard architecture

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures: the Harvard architecture

Von Neumann vs. Harvard Architecture

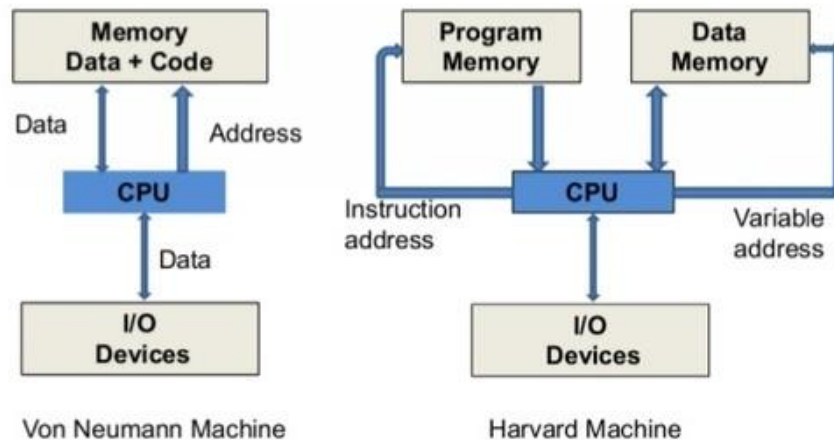
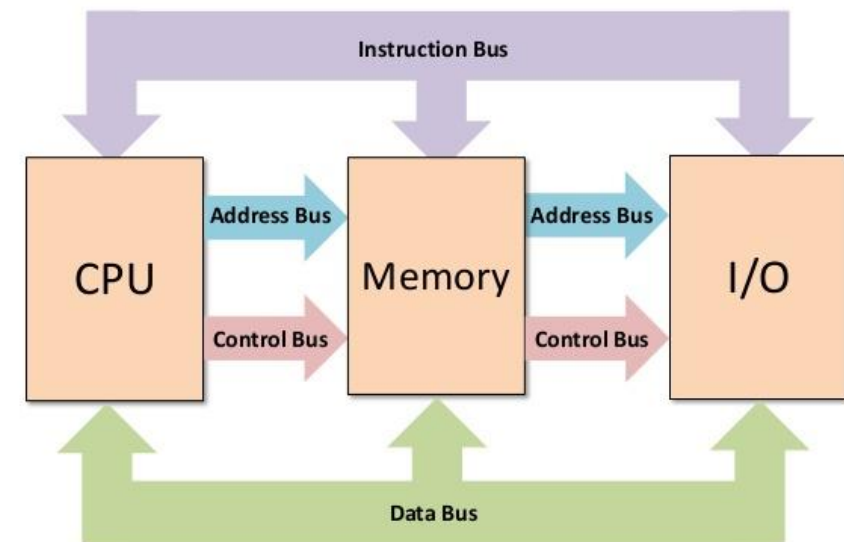


Figure 7.5: von Neumann vs Harvard architecture

Harvard Architecture



ECE @ Saint Louis University, Baguio City

6

Figure 7.6: Harvard architecture

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- **The Harvard architecture**
- The Harvard architecture potentially provides a **higher performance** level by **parallelizing accesses** to **instructions** and **data**.
- This architecture also removes the entire class of **security** issues associated with maliciously executing data as program instructions, *as long as the **instruction memory** cannot be modified by program instructions*.
- *This assumes that the **program memory** is loaded with instructions in a trustworthy manner.*
- In hindsight, with knowledge of the proliferation of **von Neumann architecture-enabled security threats**, there is reason to wonder whether the entire information technology industry would not have been vastly better off had there been early agreement to embrace the Harvard architecture and its complete separation of code and data memory regions, despite the costs involved.
- In practice, a **strict Harvard architecture** is **rarely used** in modern computers. Several variants of the Harvard architecture are commonly employed, under the name **modified Harvard architecture**.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- **The modified Harvard architecture**
- Computers designed with a **modified Harvard architecture** have, in general, **some degree of separation** between **program instructions** and **data**. This separation is rarely absolute.
- While systems with modified Harvard architectures contain separate program instruction and data memory regions, such systems typically support some means of storing data in program memory and, in some cases, storing instructions in data memory.
- The following diagram shows a typical modified Harvard architecture representing a variety of real-world computer systems:

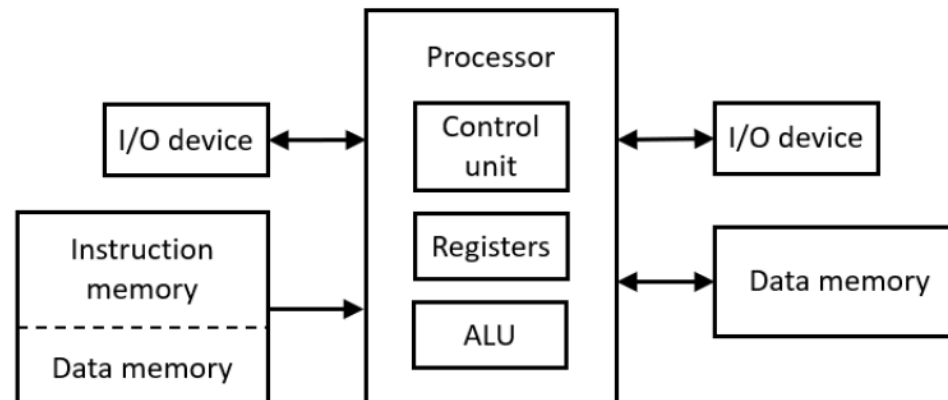


Figure 7.7: Modified Harvard architecture

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- The modified Harvard architecture
- Digital Signal Processors (DSPs) achieve substantial benefits from the use of a Harvard-like architecture.
- By storing one numeric vector in *instruction memory* and a second vector in *data memory*, a DSP can execute one **multiply-accumulate** (MAC) operation per processor clock cycle.
- In these systems, *instruction memory* and the *data elements* it contains are typically *read-only* memory devices.
- This is indicated by the **unidirectional arrow** connecting the instruction memory to the processor in **Figure 7.7**. Consequently, *only constant data values* are suitable for storage in the instruction memory region.
- Besides DSPs, most modern general-purpose processors contain **separate instruction and data caches**, thereby implementing significant aspects of the Harvard architecture.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures

- **The modified Harvard architecture**
- Processor architectures such as **x86** and **ARM** support **parallel, independent access** to **instructions** and **data** when the requested items happen to reside in the **first level of cache memory**.
- When **on-chip cache** lookups are unsuccessful, the processor must access the main memory over the von Neumann-style shared bus, which takes significantly longer.
- As a practical matter, the implementation details of a particular processor in terms of von Neumann-versus-Harvard architectural features seldom matter to software developers, other than in terms of performance considerations.
- *Programmers generally develop programs in their high-level language of choice and the compiler or interpreter handles the details related to allocating data and instructions to the appropriate memory regions.*

— Computer Architecture

Chapter 7: Processor and Memory Architectures

1. The von Neumann, Harvard and modified Harvard architectures: the modified Harvard architecture

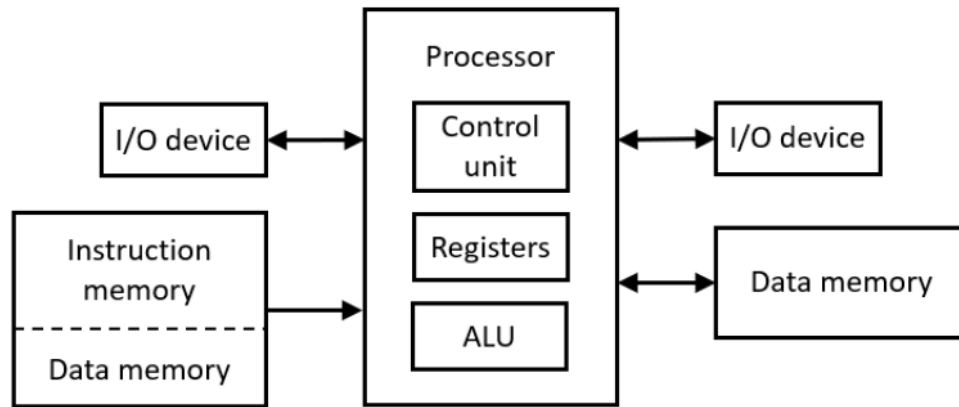


Figure 7.8: modified Harvard architecture

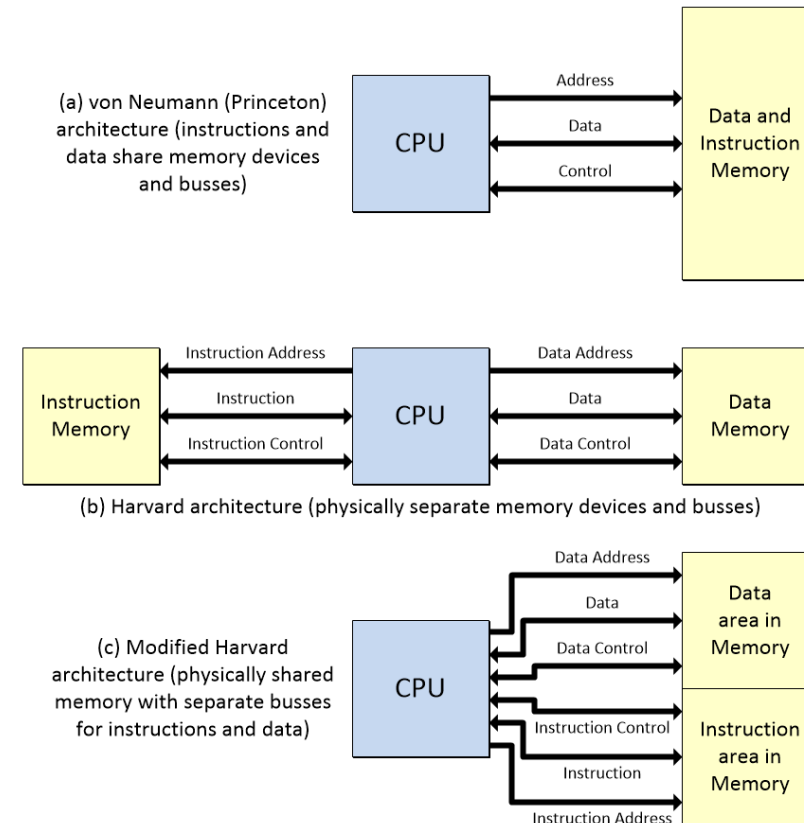


Figure 7.9: von Neumann vs Harvard vs modified Harvard architectures

— Computer Architecture

Chapter 7: Processor and Memory Architectures

Contents

1. The von Neumann, Harvard and modified Harvard architectures
- 2. Physical and virtual memory**
3. Paged virtual memory
4. Memory management unit

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory

- Memory devices in computers can be categorized as **random-access memory (RAM)**, which can be **read** from and **written** to at will, and **read-only memory (ROM)**, which, as the name indicates, can be **read** but **not written**.
- Some types of memory devices, such as **flash memory** and **electrically erasable programmable read-only memory (EEPROM)**, inhabit a middle ground, where the data content of the devices can be changed, just not as easily, or as quickly, or updated such a large number of times, as standard RAM.
- Memory devices within a computer must be configured **to ensure that each device occupies a unique span of the system address space**, enabling the processor to access each of possibly several RAM and ROM devices by setting its address lines appropriately.
- **Modern computer systems** generally perform this **address space allocation** *automatically*, based on the slot a memory device occupies.
- Software running on early computer systems, and on the **less-sophisticated computers** and **embedded processors** of today (such as 6502-based systems), uses the **addresses** of RAM and ROM devices in program instructions to perform reads and writes.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory

- In systems using **physical addressing**, the memory addresses in instructions are the actual addresses of the referenced instruction or data item.
- This means the memory address contained in an instruction is the same address used to electrically access the appropriate location in a memory device.
- This architectural approach is conceptually straightforward to implement in processor designs, but in any application scenario involving multiple **simultaneously** executing programs (referred to as **multiprogramming**), the burden of software development can become excessive.
- If each one of multiple programs is developed in **isolation** from the others (in a scenario with multiple independent developers, for example), there must be some way to **allocate** the available **RAM** and **ROM address spaces to individual programs** so that multiple programs can be in the running state simultaneously (perhaps in the context of an RTOS) without interfering with each other's use of memory.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory

- One well-known early effort to support the execution of multiple programs in a single address space is **the MS-DOS terminate and stay resident (TSR) program concept**.
- **TSR programs** allocate memory and load their code into it, and then return control to the operating system. Users can continue to work with the system normally, loading and using other applications (one at a time, of course), but they can also access the TSR as needed, typically by typing **a special key combination**.
- It is possible to load multiple TSR programs in memory **simultaneously**, each accessible via *its own key combination*. After activating a TSR program, the user interacts with it as needed, and then executes a TSR command to return to the currently running main application.
- While limited in many ways (including consuming a portion of the maximum of **640 KB** of RAM available in those early PCs), *TSR programs effectively enabled the execution of multiple programs in a single RAM address space*.
- Developing TSR applications was a challenging task, and the more advanced TSR programs available in the 1980s and 1990s took advantage of undocumented MS-DOS features in order to provide maximum utility to their users.
- As a result of this complexity, TSR programs developed a reputation for causing system **instability**. A different approach for supporting multiprogramming was clearly needed.

Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory: MS DOS TPA (Transient Program Area)

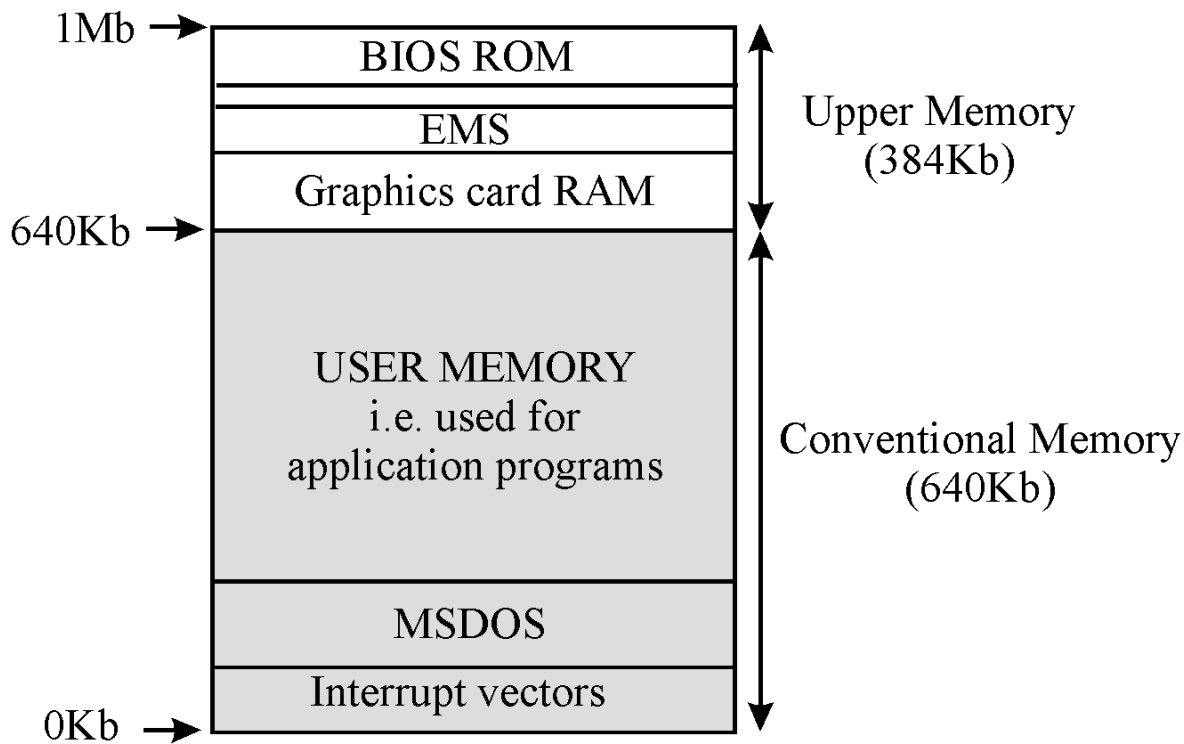


Figure 7.10: MS DOS memory

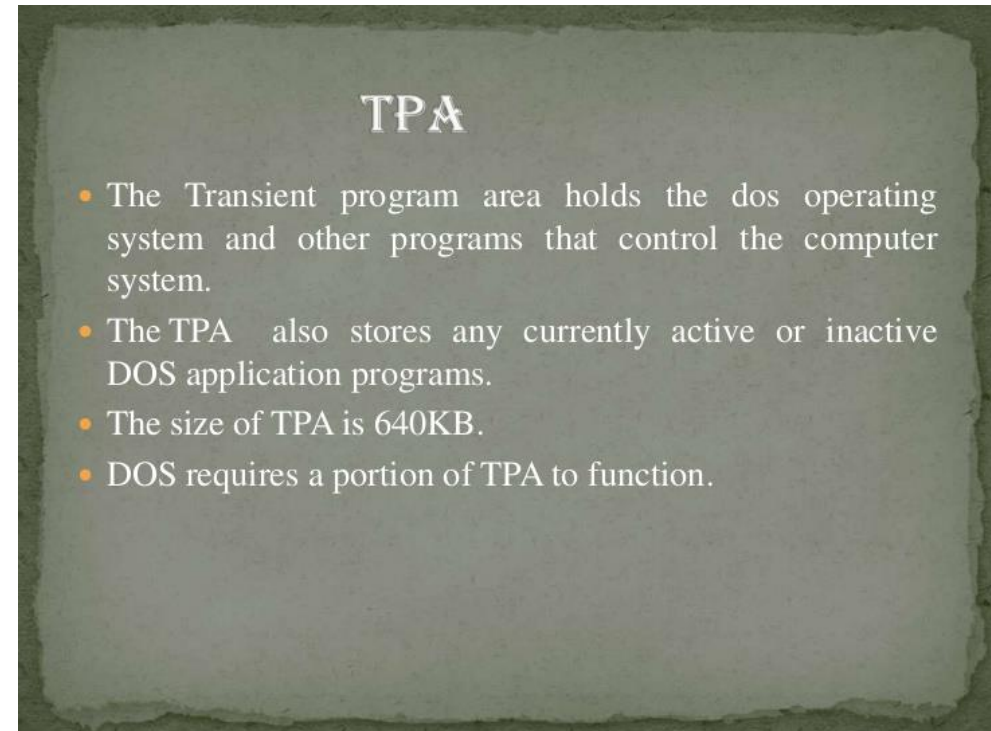


Figure 7.11: MS-DOS Transient Program Area (TPA)

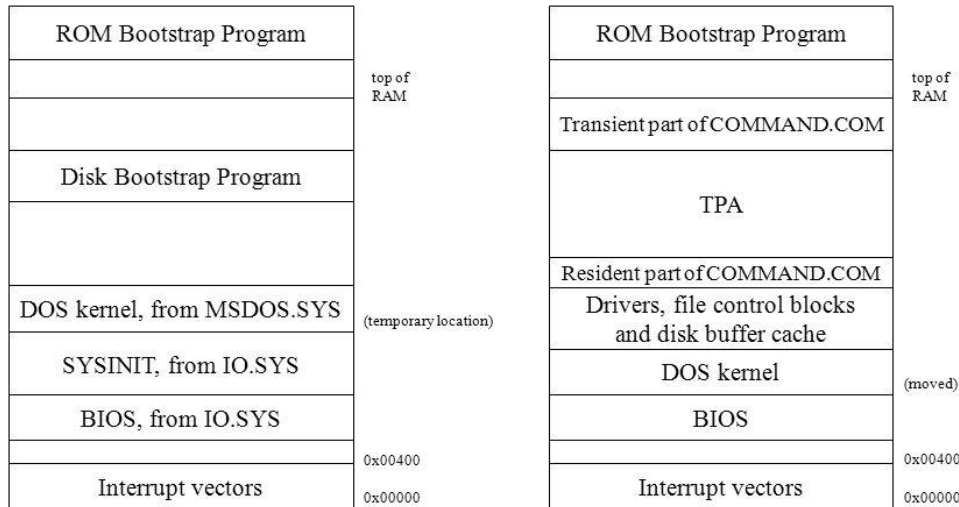
— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory: MS DOS TPA (Transient Program Area)

DOS Memory Organization and OS Loading

Diagrams derived from Ray Duncan, *Advanced MS-DOS Programming*



Overlays in MS-DOS by Andrew C. Vogan

6

Figure 7.12: MS DOS memory organization

FREE TPA

- This area holds DOS application programs as they are executed. This application includes word processor and spreadsheets. Also it holds TSR (**terminate and stay resident**) programs that remain in memory in an inactive state until activated by interrupt.

Figure 7.13: terminate and stay resident (TSR)

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory

- The use of **virtual memory** overcomes the biggest challenges that prohibited the widespread use of **multiprogramming** in the original PC design.
- Virtual memory is a **method of memory management** that enables each application to operate in its own memory space, seemingly independent of any other applications that may be in the running state simultaneously on the same system.
- In a computer with **virtual memory management**, the **operating system** is *responsible* for the allocation of physical memory to system processes and to user applications.
- The **memory management** hardware and software **translate** memory requests originating in the application's **virtual memory** context to **physical memory** addresses.
- *Apart from easing the process of developing and running concurrent applications, virtual memory also enables the allocation of a larger amount of physical memory than actually exists in the computer.*
- This is possible through the use of **secondary storage** (typically a *disk file*) to temporarily hold copies of sections of memory removed from physical memory to allow a different program (or a different part of the same program) to run in the now-free memory.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory

- In modern general-purpose computers, memory sections are usually allocated and moved in multiples of a fixed-size chunk, called a **page**.
- Memory pages are typically **4 KB or larger**.
- Moving memory pages to and from secondary storage in virtual memory systems is called **page swapping**. The file containing the swapped-out pages is the **swap file**.
- In a virtual memory system, neither application developers nor the code itself need to be concerned about how many other applications are running on the system or how full the physical memory may be getting.
- As the application allocates memory for data arrays and calls library routines (which requires the code for those routines to be loaded into memory), *the operating system manages the use of physical memory and takes the steps necessary to ensure that each application receives memory upon request.*
- Only in the unusual case of completely filling the available physical memory while also filling the swap file to its limit is the system forced to return a *failure code* in response to a memory allocation request.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory: MS DOS TSR (terminate and stay resident)

Virtual Memory

- Two kinds of addresses:
 - CPU uses **virtual addresses**
 - Main memory uses **physical addresses**
- Hardware translates virtual addresses to physical addresses via an operating system (OS)-managed table, the **page map**

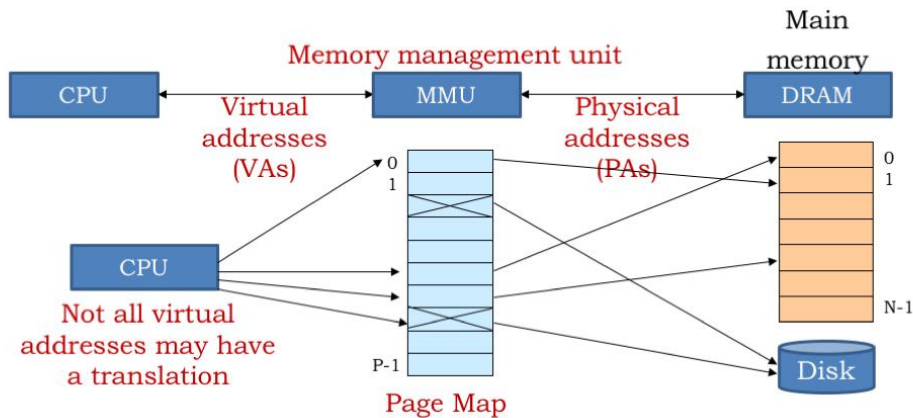


Figure 7.14: physical & virtual memory

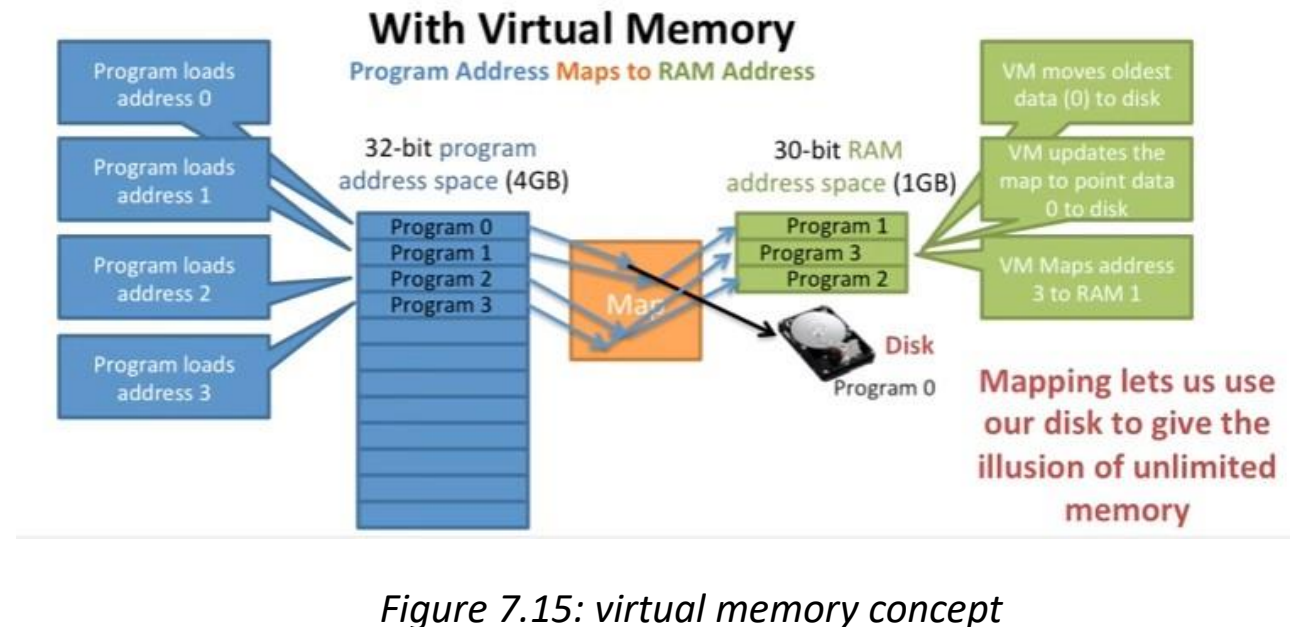


Figure 7.15: virtual memory concept

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory

- Virtual memory provides several notable **benefits** besides making things easier for programmers:
 - Not only are applications able to ignore each other's presence, they are **prevented from interfering with each** other, accidentally or intentionally.
 - The virtual memory management hardware is responsible for ensuring that each application can only access memory pages that have been assigned to it.
 - Attempts to access another process's memory, or any other address outside its assigned memory space, result in an **access violation exception**.
 - Each memory page has a **collection of attributes** that restrict the types of operations supported within it.
 - A page may be marked **read-only**, causing any attempts to write data to the page to fail.
 - A page may be marked **executable**, meaning it contains code that can be executed as processor instructions.
 - A page may be marked **read-write**, indicating the application is free to modify the page at will.
 - By setting these attributes appropriately, operating systems can *improve stability* by ensuring that instructions can't be modified and that the execution of data as instructions cannot occur, whether such an attempt is the result of an accident or malicious intent.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory

- Virtual memory provides several notable **benefits** besides making things easier for programmers:
 - Memory pages can be marked with a **minimum required privilege level**, allowing pages to be restricted for access only by code running with **kernel privilege**.
 - This restriction ensures that the operating system continues operating properly even in the presence of misbehaving applications.
 - This allows system memory to be mapped into each process's address space while prohibiting application code from interacting directly with that memory.
 - *Applications can only access system memory indirectly, via a programming interface consisting of system function calls.*
 - Memory pages can be marked as **shareable among applications**, meaning a page can be explicitly authorized as accessible from more than one process. This enables efficient **interprocess communication**.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory

- Early versions of **Microsoft Windows** implemented some features of **memory virtualization** using 80286 and 80386 processor **memory segmentation capabilities**.
- In the Windows context, the use of virtual memory came into its own with the introduction of **Windows NT 3.1** in **1993**.
- The **Windows NT** system architecture was based on the Digital Equipment Corporation **Virtual Address Extension (VAX) architecture**, developed in the 1970s.
- The **VAX** architecture implemented a **32-bit virtual memory environment** with a **4 GB virtual address space** available to each of potentially many applications running in a **multiprogramming** context.
- One of the key architects of the VAX operating system, Virtual Memory System (VMS), was **David Cutler**, who later led the development of **Microsoft Windows NT**.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

2. Physical and virtual memory

- **Windows NT** has a **flat 32-bit memory organization**, meaning any address in the entire 32-bit space is accessible using a 32-bit address. No additional programmer effort is required to manipulate segment registers.
- By default, the Windows NT virtual address space is divided into two equal-sized chunks: a **2 GB user address space** in the **lower** half of the range, and a **2 GB kernel space** in the **upper** half.
- The next section delves into the implementation of paged virtual memory in **32-bit Windows NT** on **Intel** processors in some detail.
- While **Windows NT** is *not entirely representative* of virtual memory implementations in other operating systems, *similar general principles apply* even if other environments differ in the details.
- This introduction will provide background on the concepts of virtual memory, while holding off on additional details related to more modern architectures, such as **64-bit processors** and **operating systems**.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

Contents

1. The von Neumann, Harvard and modified Harvard architectures
2. Physical and virtual memory
- 3. Paged virtual memory**
4. Memory management unit

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

- In **32-bit Windows NT** on **Intel** processors, memory **pages** are **4 KB** in size.
- This implies that addressing a location within a particular page requires **12 address bits** ($2^{12} = 4096$). The remaining **20 bits** of a 32-bit virtual address are used in the virtual-to-physical translation process.
- In **Windows NT**, all memory addresses in a **program** (both in the source code and in compiled executable code) are **virtual addresses**. They are not associated with physical addresses until the program runs under the control of the **memory management unit**.
- A contiguous 4 KB section of Windows NT **physical** memory is called a **page frame**. The page frame is the smallest unit of memory managed by the Windows virtual memory system.
- Each page frame starts on a **4 KB boundary**, meaning the lower 12 address bits are all **zero** at the base of any page frame.
- The system tracks information related to page frames in **page tables**.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

- A **Windows NT page table** is sized to occupy a single **4 KB page**.
- Each 4-byte entry in a page table enables the translation of a 32-bit address from the virtual address space used by program instructions to a physical address required to access a location in RAM or ROM.
- A 4 KB **page table** contains **1,024 page address translations**.
- A **single page table** manages access to **4 MB of address space**: each page table contains 1,024 page frames multiplied by 4 KB per page.
- A process may have several associated page tables, all of which are managed by a **page table directory**.
- A page table directory is a 4 KB page containing a series of 4-byte references to page tables.
- A **page table directory** can contain **1,024 page table references**.
- A single page table directory covers the entire **4 GB address space** (4 MB per page table multiplied by 1,024 page table references) of **32-bit Windows NT**.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

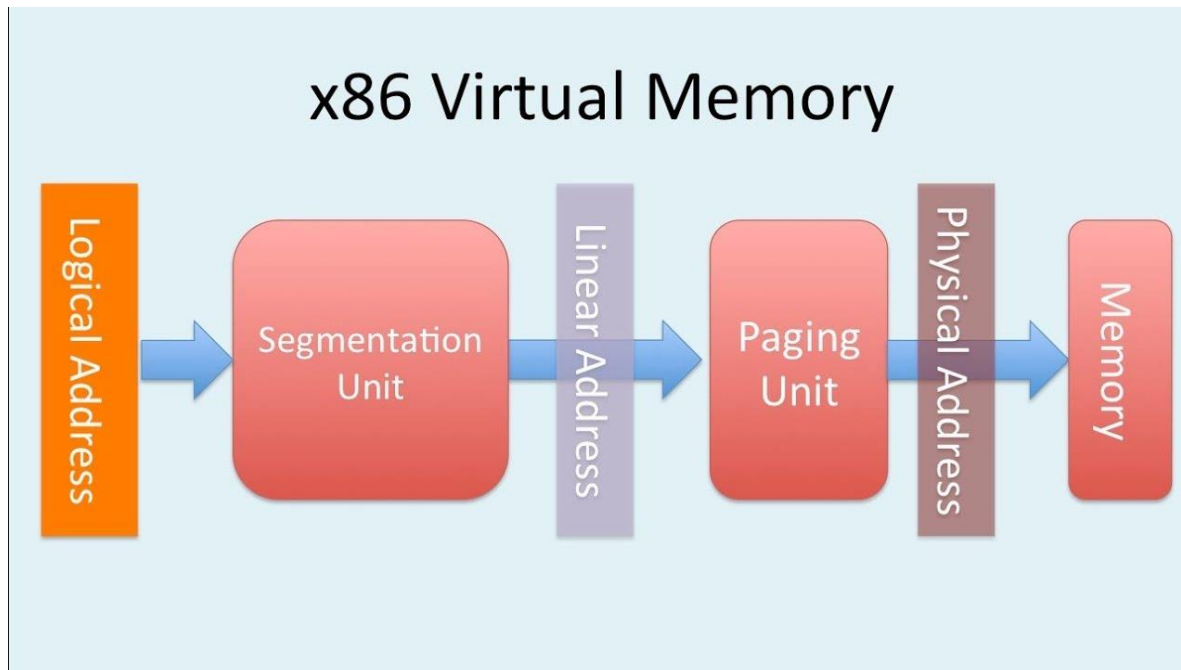


Figure 7.16: x86 Virtual Memory

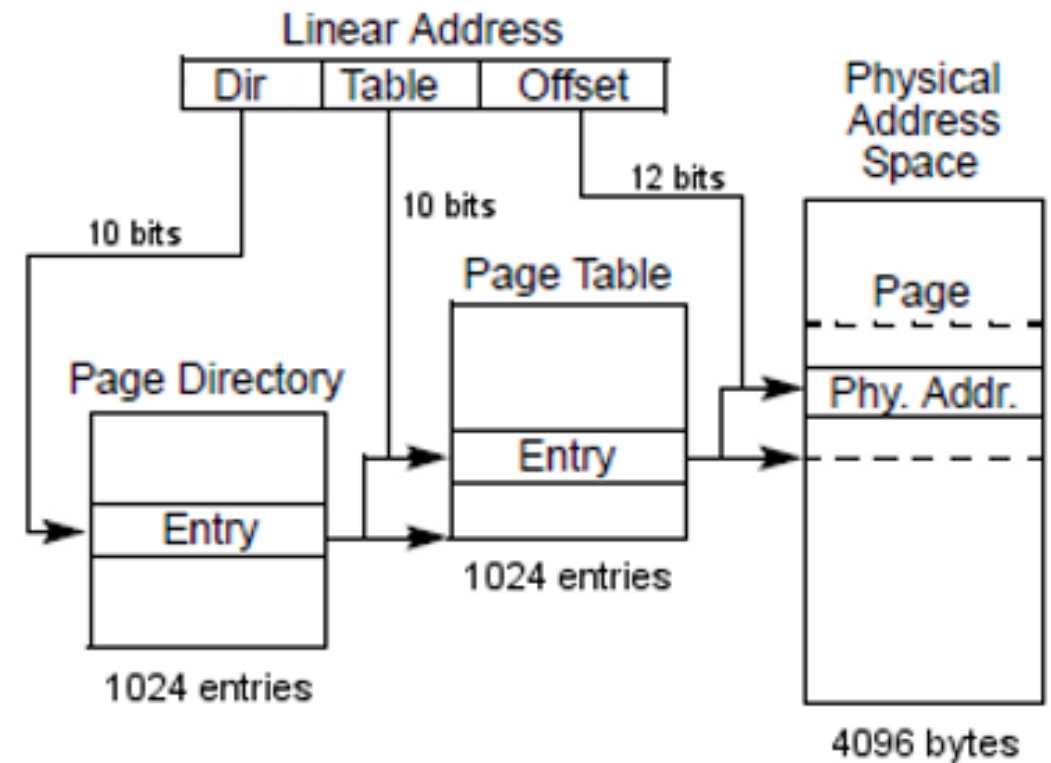


Figure 7.17: x86 memory page memory management

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

- Each Windows NT process has its **own page table directory**, **set of page tables**, and **collection of page frames** allocated for its use. The process page tables apply to all threads within the process because all of a process's threads share the same address space and memory allocations.
- *When the system scheduler switches from one **process** to another, the virtual memory context of the incoming process replaces the context of the outgoing process.*
- Intel x86 processors maintain the address of the current process page table directory in the **CR3 register**, also known as the **Page Directory Base Register (PDBR)**.
- This single entry point to the page table directory, and indirectly to the page tables, enables the processor to translate any valid virtual address to the corresponding physical address.
- In accessing an arbitrary valid location in memory, and assuming information to expedite the access is not already stored in cache memory, the processor first looks up the physical address of the relevant page table in the page table directory using the upper 10 bits of the virtual address.
- It then accesses the page table and uses the next most significant 10 address bits to select the physical page containing the requested data.
- The lower 12 bits of the address then specify the memory location in the page frame requested by the executing instruction.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

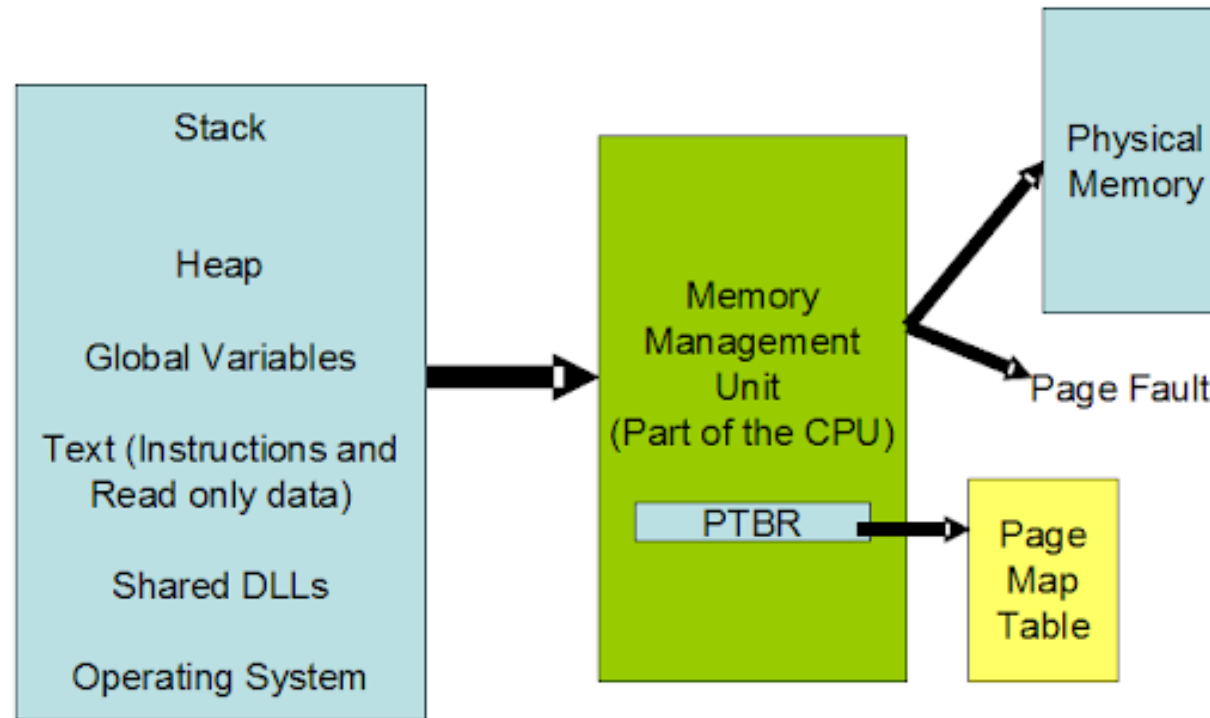


Figure 7.18: paged virtual memory

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

Page frames do not represent actual divisions in physical memory

Physical memory is not actually divided into page frames. The page structure is merely a method that the system uses to keep track of the information required to translate virtual addresses to physical memory locations.

- To meet users' performance needs, memory access must be as fast as possible. At least one virtual-to-physical translation must take place during the execution of every instruction to fetch instruction opcodes and data.
- Due to the high-frequency repetition of this process, processor designers expend an extraordinary amount of effort ensuring that the process of virtual address translation takes place as efficiently as possible.
- In modern processors, **cache memory** retains the results of **recent virtual memory translation lookups**. When implemented effectively, this approach enables a very high percentage of virtual memory translations to occur internally in the processor, without any of the clock cycles that would be required if the processor needed to look up a page table address in the page table directory and then access the page table to determine the requested physical address.
- The data structures used in virtual-to-physical address translations are not accessible to applications running at user privilege level. *All of the activity related to address translation takes place in processor hardware and in kernel mode software processes.*

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

- To help clarify the use of virtual memory, the following diagram presents an example of how Windows translates a 32-bit virtual address to a physical address:
- We'll go through the translation process in **Figure 7.19** step by step.
- Assume the processor is requesting the 8-bit data value stored at virtual address **\$00402003** with an instruction such as **mov al, [ebx]** where **ebx** has previously been loaded with the value **\$00402003**.
- We will assume the translation for this address is not already stored in the processor cache, and we'll also assume that the page is resident in the main memory.*

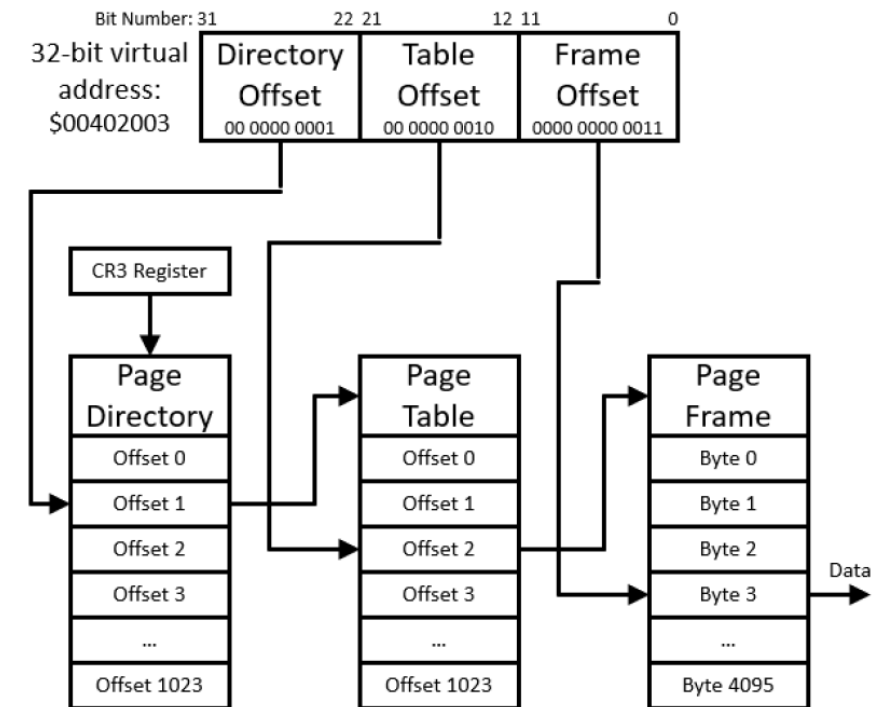


Figure 7.19: Virtual to physical address translation

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

- The following procedure describes the translation process:
 - The processor attempts to execute the `mov al, [ebx]` instruction, but it cannot complete it because it does not have immediate access to the information needed to perform the translation of the **virtual address** in **ebx** to the corresponding **physical address**. This generates a **page fault**, which transfers control to the **operating system** so that it can **resolve the address translation**. The use of the term fault here does not imply that some kind of error occurred. *Page faults are a routine part of application execution.*
 - The requested virtual address is **shifted right** by **22 bit positions**, leaving the 10-bit directory offset, which has the **value 1** in this example.

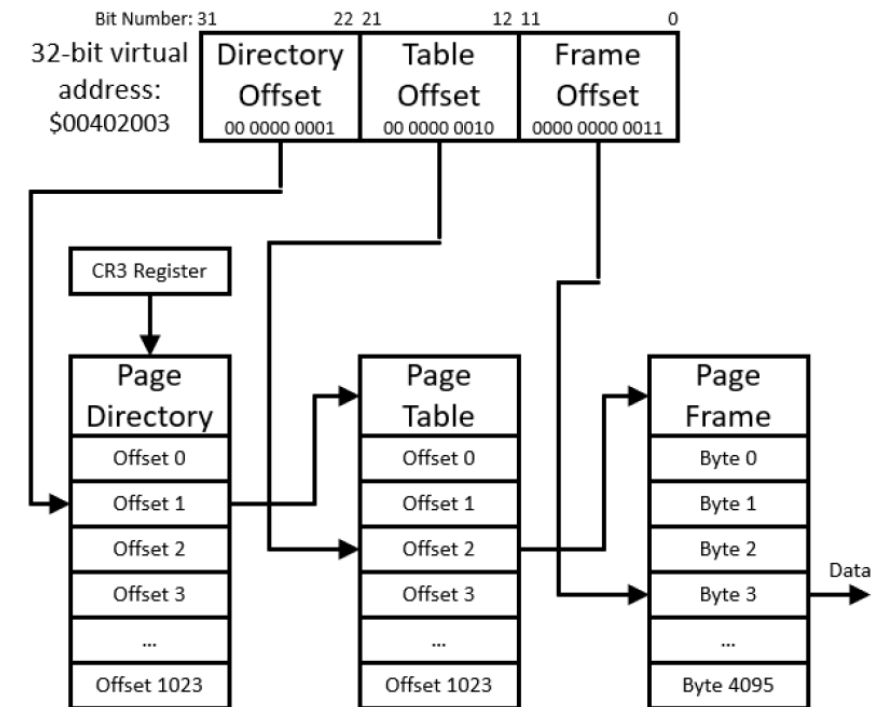


Figure 7.19: Virtual to physical address translation

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

- The following procedure describes the translation process:
- The **directory offset** is **shifted left** by **2 bit positions** (because each entry in the page directory is 4 bytes) and is added to the content of processor register **CR3** (the PDBR). The result is the address of the page table directory entry containing the address of the relevant page table.
 - The **requested virtual address** is **shifted right** by **12 bit positions**, and masked to leave only the **10-bit table offset**, which has the **value 2** in this example.

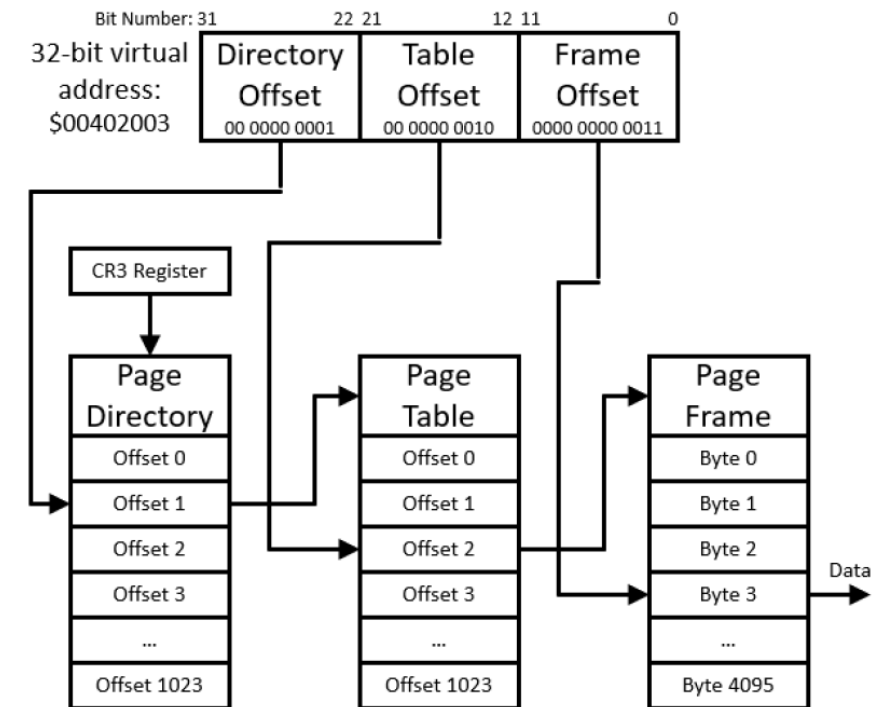


Figure 7.19: Virtual to physical address translation

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

- The following procedure describes the translation process:
- The **table offset** is **shifted left** by **2 bit positions** (because each entry in this table is also 4 bytes) and added to the page table address identified in Step 3. The 32-bit address read from this location is the **physical** address of the page frame containing the requested data.
 - The processor stores the translation, which is a conversion from the upper 20 bits of a virtual address to the corresponding upper 20 bits of a page frame address, in its **translation cache**.
 - The processor restarts the `mov al, [ebx]` instruction, which will now succeed in moving the requested data byte into the `al` register using the **cached virtual-to-physical translation**. The lower 12 bits of the virtual address (the **frame offset**), which contain the **value 3** in this example, are added to the page frame address computed in **Step 5** to access the requested byte.*

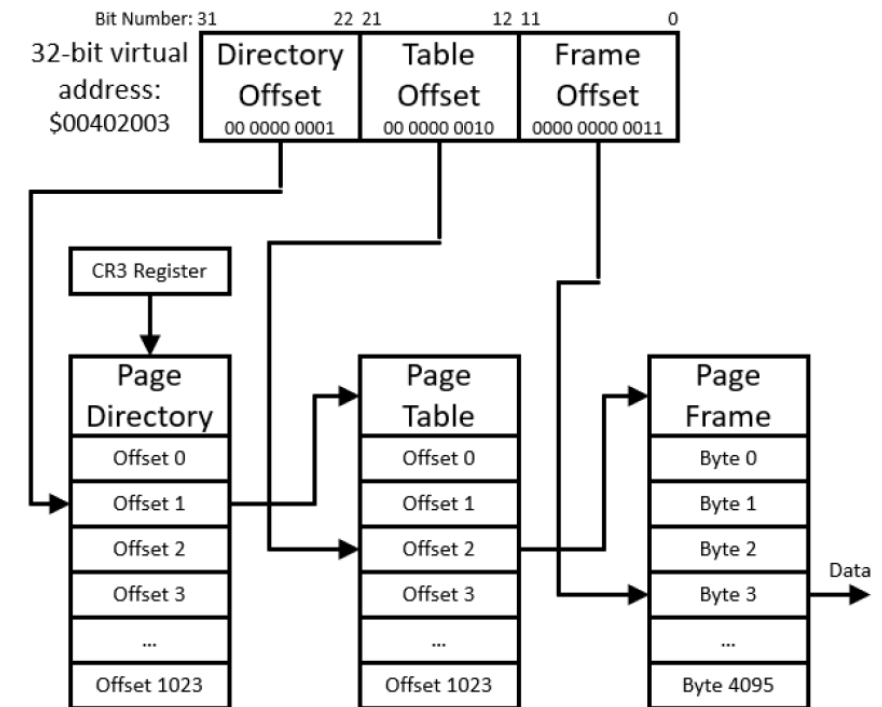


Figure 7.19: Virtual to physical address translation

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory

- Once these steps are complete, the translation for the requested page frame remains available in the processor **cache memory**.
- *Subsequent requests for **the same virtual address or for other locations in the same page frame** will execute without delay until the cache entry for this page frame is overwritten by code executing at a later time.*
- The page fault procedure described in the preceding steps is a **soft fault**, which sets up the virtual-to-physical translation for a page that is already accessible by the processor *but that is not in the translation cache*.
- A **hard fault** occurs when accessing a page that has been **swapped to secondary storage**.
- Processing a **hard fault** requires several additional steps, including allocating a page frame to receive the requested frame, requesting the page from secondary storage, and updating the page table with the physical address of the page.
- Because **hard faults** involve a **disk transfer**, this type of fault has a much greater impact on application performance than a **soft fault**.
- The translation process converts the upper 20 bits of a virtual address to the corresponding 20 bits of the physical address. This leaves 12 bits in each page table entry available to contain status and configuration information for the page frame.
- The use of these bits is described in the following section.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory: Page status bits

- The following table describes each of the **12 status bits** in a **32-bit Windows NT page table entry**.
- The **processor** uses the *page status bits* to maintain status information and to control access to each page by system and user processes.
- The **Owner** bit identifies a page as *owned* by the **kernel** or by a **user**. *User processes cannot read or write any pages owned by the kernel*. Any attempt to write to a page marked **read-only** (where the page's **Write** bit is 0) results in an **access violation exception**.

Bit	Name	Description
0	Valid	1 indicates that this page table entry is usable for translation. If this bit is 0, the remaining bits may have different meanings, as defined by the operating system. The following bit descriptions assume that the valid bit is 1.
1	Write	1 indicates that the page is writeable. 0 means that the page is read-only.
2	Owner	1 indicates that the page is user mode. 0 indicates that the page is kernel mode.
3	Write through	1 indicates that changes to the page are to be flushed to disk immediately. 0 indicates that page changes will be maintained in RAM.
4	Cache disabled	1 indicates that caching is disabled for the page. 0 means that caching is enabled.
5	Accessed	1 indicates that the page has been read or written. 0 indicates the page has not been accessed in any way.
6	Dirty	1 indicates that the page has been written. 0 indicates that no writes have occurred.
7	Reserved	Unused.
8	Global	1 indicates that this translation applies to all processes. 0 means that this entry applies to only one process.
9	Reserved	Unused.
10	Reserved	Unused.
11	Reserved	Unused.

Table 7.1: Page status bits

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory: Page status bits

- The system uses the page status bits to manage memory as efficiently as possible.
- If the **Accessed** bit is **clear**, the page has been allocated but never used.
- *When the system needs to free physical memory, pages that have never been accessed are prime candidates for removal because there is no need to save their contents when removing them from memory.*
- Similarly, if a page's **Dirty** bit is **clear**, the page has not had its contents modified since it was brought into memory.
- *The memory manager can release pages that do not have the **Dirty** bit set with the knowledge that when the page is needed again, it can be reloaded from its source location (typically a disk file) to restore it accurately.*

Bit	Name	Description
0	Valid	1 indicates that this page table entry is usable for translation. If this bit is 0, the remaining bits may have different meanings, as defined by the operating system. The following bit descriptions assume that the valid bit is 1.
1	Write	1 indicates that the page is writeable. 0 means that the page is read-only.
2	Owner	1 indicates that the page is user mode. 0 indicates that the page is kernel mode.
3	Write through	1 indicates that changes to the page are to be flushed to disk immediately. 0 indicates that page changes will be maintained in RAM.
4	Cache disabled	1 indicates that caching is disabled for the page. 0 means that caching is enabled.
5	Accessed	1 indicates that the page has been read or written. 0 indicates the page has not been accessed in any way.
6	Dirty	1 indicates that the page has been written. 0 indicates that no writes have occurred.
7	Reserved	Unused.
8	Global	1 indicates that this translation applies to all processes. 0 means that this entry applies to only one process.
9	Reserved	Unused.
10	Reserved	Unused.
11	Reserved	Unused.

Table 7.1: Page status bits

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory: Page status bits

- *Pages that have the **Dirty** bit **set** require storage in the **swap file** when they are removed from memory.*
- When a page is moved to the swap file, the page table entry is updated in a different format from Table 7.1 to indicate that it is not valid for translation (the **Valid** bit is **clear**) and to store its location within the swap file.
- The format for valid page table entries is defined by the processor architecture, in this case, the Intel x86 family.

Bit	Name	Description
0	Valid	1 indicates that this page table entry is usable for translation. If this bit is 0, the remaining bits may have different meanings, as defined by the operating system. The following bit descriptions assume that the valid bit is 1.
1	Write	1 indicates that the page is writeable. 0 means that the page is read-only.
2	Owner	1 indicates that the page is user mode. 0 indicates that the page is kernel mode.
3	Write through	1 indicates that changes to the page are to be flushed to disk immediately. 0 indicates that page changes will be maintained in RAM.
4	Cache disabled	1 indicates that caching is disabled for the page. 0 means that caching is enabled.
5	Accessed	1 indicates that the page has been read or written. 0 indicates the page has not been accessed in any way.
6	Dirty	1 indicates that the page has been written. 0 indicates that no writes have occurred.
7	Reserved	Unused.
8	Global	1 indicates that this translation applies to all processes. 0 means that this entry applies to only one process.
9	Reserved	Unused.
10	Reserved	Unused.
11	Reserved	Unused.

Table 7.1: Page status bits

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory: Page status bits

- The processor hardware directly accesses the page table entries to perform virtual-to-physical address translation and enforce page protections as the processor runs at full speed.
- *In addition to managing the memory used by each process, the system must keep track of all of the **RAM** and **ROM** page frames in the computer, whether in use by a process or not.*
- The system maintains this information in lists called **memory pools**, described next.

Bit	Name	Description
0	Valid	1 indicates that this page table entry is usable for translation. If this bit is 0, the remaining bits may have different meanings, as defined by the operating system. The following bit descriptions assume that the valid bit is 1.
1	Write	1 indicates that the page is writeable. 0 means that the page is read-only.
2	Owner	1 indicates that the page is user mode. 0 indicates that the page is kernel mode.
3	Write through	1 indicates that changes to the page are to be flushed to disk immediately. 0 indicates that page changes will be maintained in RAM.
4	Cache disabled	1 indicates that caching is disabled for the page. 0 means that caching is enabled.
5	Accessed	1 indicates that the page has been read or written. 0 indicates the page has not been accessed in any way.
6	Dirty	1 indicates that the page has been written. 0 indicates that no writes have occurred.
7	Reserved	Unused.
8	Global	1 indicates that this translation applies to all processes. 0 means that this entry applies to only one process.
9	Reserved	Unused.
10	Reserved	Unused.
11	Reserved	Unused.

Table 7.1: Page status bits

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory: Memory pools

- Windows NT categorizes **memory pools** into two types: **non-paged** and **paged**.
 - **Non-paged pool:**
 - The non-paged pool contains all page frames that are guaranteed to **remain resident in memory at all times**.
 - Code for **interrupt service routines, device drivers**, and the **memory manager itself** must always remain directly accessible by the processor for system performance reasons and, in the case of the memory manager itself, for the system to function at all.
 - **Non-paged virtual addresses reside in the system portion of a process's virtual address space.**
 - **Paged pool:**
 - The paged pool contains virtual memory pages that **can be temporarily swapped out of physical memory** as needed.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory: Memory pools

- The system tracks the status of every page frame in *physical memory* in a structure called the **page frame number (PFN) database**. The **PFN** is the **upper 20 bits of the physical base address of a page frame**.
- Each page frame can be in one of several states depending on its current and previous usage.
- These are some of the key page frame states:
 - **Active**: The page frame is **part of a system or user process working set**. A **working set** is the portion of a process's virtual address space *currently present in physical memory*.
 - **Standby**: Standby pages have been **removed from process working sets** and have **not been modified**.
 - **Modified**: Modified pages have been **removed from process working sets** and **have been modified**. These pages **must be written to disk before** their page frames are **reused**.
 - **Free**: Free pages are **unused** but still contain data from their last membership in a working set. For security reasons, these pages **cannot be made available** to a user process **until** their **data content has been overwritten with zeros**.
 - **Zeroed**: Zeroed pages are free and have been **overwritten with zeros**. These pages are **available** for allocation by user processes.
 - **Bad**: Bad pages have generated **hardware errors** during processor accesses. *Bad pages are tracked in the PFN database and are not used by the operating system.*

— Computer Architecture

Chapter 7: Processor and Memory Architectures

3. Paged virtual memory: Memory pools

- As system services and applications start up, run, and shut down, page frames transition between states under system control. In **Windows NT**, a **system task** runs during *idle periods* and converts **free pages** to **zeroed pages** by overwriting those pages with zeros.
- This discussion has focused on the implementation of virtual memory in the x86 processor architecture under Windows NT.
- Other processor architectures and operating systems implement virtual memory using similar concepts.
- The processor component that controls the **memory allocation**, **address translation**, and **protection functions** is called a **memory management unit**.
- The next section examines the **memory management unit (MMU)** as a generalized computer system component.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

Contents

1. The von Neumann, Harvard and modified Harvard architectures
2. Physical and virtual memory
3. Paged virtual memory
4. **Memory management unit**

— Computer Architecture

Chapter 7: Processor and Memory Architectures

4. Memory management unit

- Processor architectures supporting **paged virtual memory** either implement the **memory management unit (MMU)** functionality **within the processor itself** or, sometimes, particularly in the case of older designs, as a **separate integrated circuit**.
- Within the MMU, the **processor's virtual address space** is divided into **page-sized allocation units**.
- Pages may be of a **fixed size**, as in the Windows NT example, or an MMU may support **multiple sizes**. Modern processors, including later generation x86 processors, often support two page sizes, one **small** and one **large**. **Small** pages are typically a few **KB** while a **large** page may be a few **MB**.
- Large page support avoids the inefficiencies associated with allocating numerous smaller pages when working with large data objects.
- *As discussed, the MMU generally contains a **cache** to improve the speed of memory access by avoiding the need to traverse the page table directory and perform a page table lookup during each memory access.*
- Although the use of **caching** for performance enhancement is a topic of *Chapter 8, Performance-Enhancing Techniques*, we'll introduce the virtual-to-physical address translation cache here because this capability is a core feature of most MMU designs.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

4. Memory management unit

- The **caching** component within the MMU that stores previously used virtual-to-physical translations is called a **translation lookaside buffer (TLB)**.
- In order to avoid looking up a page table in the page table directory, and then looking up the page frame in the referenced page table on every memory access, the **TLB stores translations** between **virtual addresses** and **page frames** resulting from those lookups in a hardware structure called an **associative memory**.

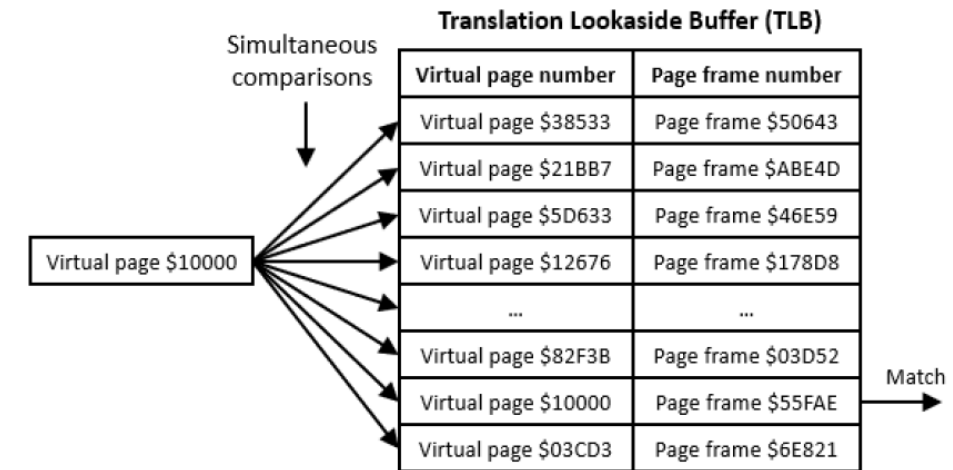


Figure 7.20: Translation lookaside buffer operation

— Computer Architecture

Chapter 7: Processor and Memory Architectures

4. Memory management unit

- *Each time the processor needs to access physical memory, which may occur multiple times during the execution of a single instruction, it first checks the TLB's associative memory to determine whether the translation information is resident in the TLB.*
- If it is, the instruction immediately uses the information stored in the TLB to access physical memory.
- If the TLB does *not* contain the requested translation, a **page fault** occurs and the processor must traverse the page table directory and a page table to determine the translated address, assuming the referenced page is resident in memory.

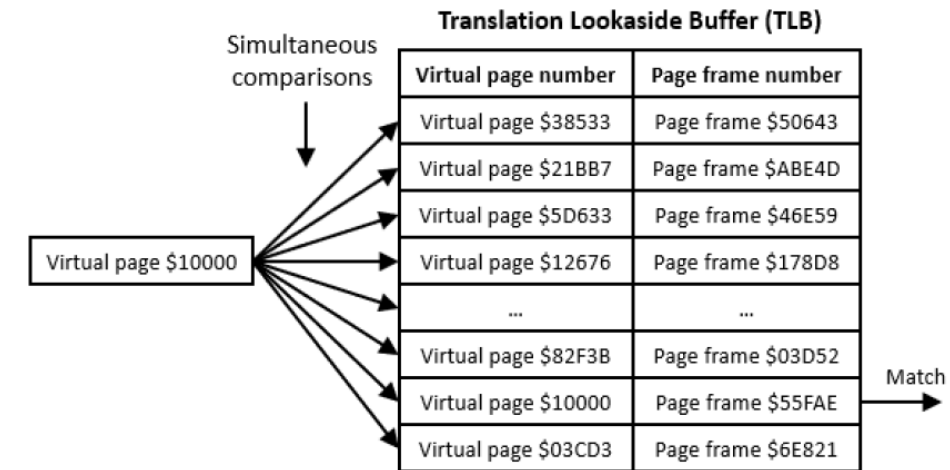


Figure 7.20: Translation lookaside buffer operation

— Computer Architecture

Chapter 7: Processor and Memory Architectures

4. Memory management unit (MMU) and Translation Lookaside Buffer (TLB)

MMU and TLB

■ Memory Management Unit (MMU)

- Hardware that translates a virtual address to a physical address
- Each memory reference is passed through the MMU
- Translate a virtual address to a physical address
 - Lots of ways of doing this!

■ Translation Lookaside Buffer (TLB)

- Cache for MMU virtual-to-physical address translations
- Just an optimization – but an important one!

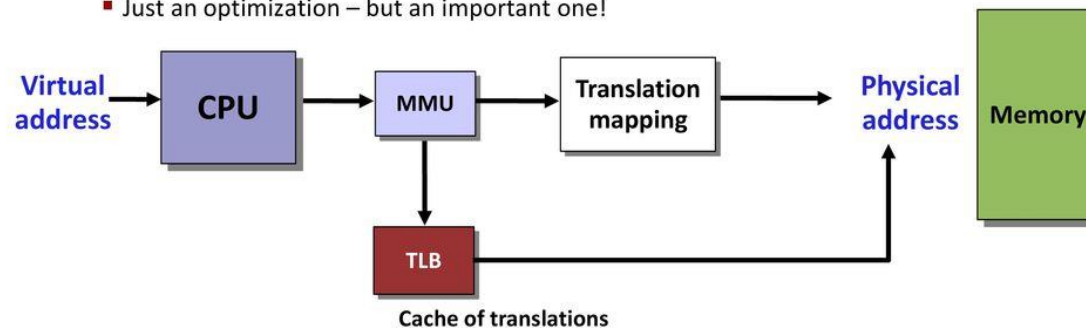


Figure 7.21: physical & virtual memory

— Computer Architecture

Chapter 7: Processor and Memory Architectures

4. Memory management unit

- On each memory access, the processor extracts the **upper 20 bits of the virtual address** to identify the **virtual page number**.
- This page number, **\$10000 (slide 54)** in this example, is used to search the **TLB** for a matching entry. The TLB hardware simultaneously compares the requested virtual page number with **all** of the virtual page numbers resident in the TLB. *If a match is found, the corresponding page frame number is immediately provided for use in accessing physical memory.*
- The TLB contains a limited number of entries, typically **64** or fewer.
- *The processor must manage which **TLB** entries are retained and which are discarded as address requests for disparate memory locations are processed.*
- **When the TLB is full, the MMU decides which of the existing TLB entries to overwrite with the new information.**
- The MMU may choose a TLB entry **at random** to replace, or a *more sophisticated implementation* may use **age information** to replace the TLB entry that has gone the longest without being used.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

4. Memory management unit

- In addition to performing virtual-to-physical address translation, **MMUs** generally perform the following functions:
 - **Separation of virtual memory into kernel space and user space:**
 - **Kernel memory** is reserved for use by the **operating system** and related components such as **device drivers**.
 - **User space** is available for use by **applications** and for other actions initiated by **users**, such as *processing commands* typed into a *command prompt window*.
 - **User-level code cannot access system memory directly**. Instead, user code must **call system functions** to request services such as the allocation of memory.
 - **Isolation of process memory:**
 - **Each process** has its **own address space**, which is the only memory it is allowed to access.
 - Unless a system-authorized memory sharing region is set up between processes, each process is prohibited from accessing memory in use by another process.
 - **One process cannot erroneously or intentionally modify memory that is private to another process.**

— Computer Architecture

Chapter 7: Processor and Memory Architectures

4. Memory management unit

- In addition to performing virtual-to-physical address translation, MMUs generally perform the following functions:
 - **Page-level access restrictions:**
 - In addition to protecting system pages from user access and protecting process-private pages from access by other processes, **a process can set protections on individual pages that it owns.**
 - Pages can be marked **read-only**, which prohibits the modification of the contents.
 - Pages marked **no-execute** cannot be used to provide instructions for processor execution.
 - In some architectures, pages can be marked **no-access**, which prohibits both reading and writing.
 - Pages containing executable code may be marked **read-only** to prevent the accidental or intentional modification of instructions in memory.

— Computer Architecture

Chapter 7: Processor and Memory Architectures

4. Memory management unit

- In addition to performing virtual-to-physical address translation, MMUs generally perform the following functions:
 - **Detection of software problems:**
 - In some programming languages, particularly the **C language**, it is unfortunately common to attempt to use a **pointer** (a pointer is a variable that contains the address of another variable) containing an **invalid address**.
 - The most common invalid address encountered in this situation is **0**, because variables are often initialized to zero. This problem is so common that the system's response to it has its own name: **the null pointer exception**.
 - When a C program attempts to access a memory location that is not in the program's valid virtual address range, such as the address \$00000000, the **MMU** triggers an **exception**, which, unless handled by the program, typically results in a **program crash** with an error message printed to the console window.
 - *In systems without virtual memory, accesses to erroneous locations may simply read or write the memory at the referenced address without any indication of an error, leading to the incorrect operation of the application or the entire system.*
 - Such **bugs** in systems **without an MMU** can be extremely difficult to fix if the problem does not become apparent immediately.

— Computer Architecture

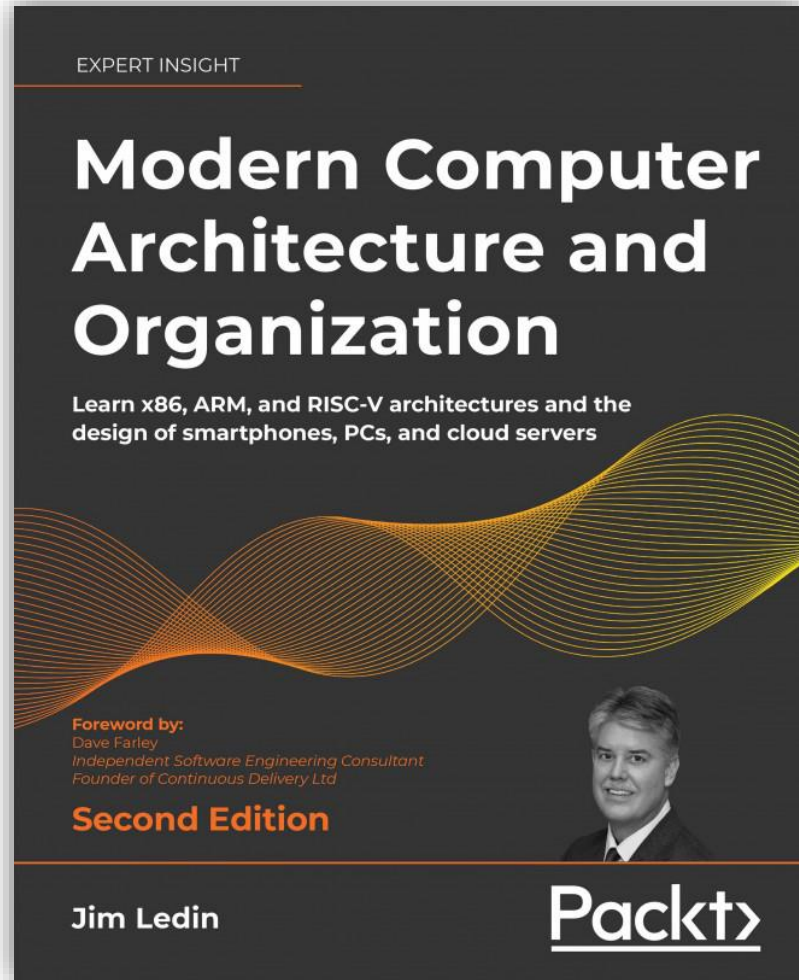
Chapter 7: Processor and Memory Architectures

4. Memory management unit

- Modern processors running **Linux**, **Windows**, and **most smart device operating systems** generally require their host systems to use **virtual memory management** and provide the **page protection mechanisms** described in this chapter.
- **Real-time embedded processors** performing safety-critical tasks such as operating **aircraft flight controls** or **managing automotive airbag operation** *may or may not* support the full feature set of an MMU.
- *One drawback related to the use of virtual memory in **hard real-time** systems is the variable time delay resulting from the need to process **soft faults** and, if **page swapping** is implemented, **hard faults**.*
- Because execution timing must be strictly controlled in many real-time systems, their designers often **avoid** the use of virtual memory.
- ***Such systems do not contain an MMU**, but they normally implement many of the other features an MMU provides, such as **hardware protection of system memory** and access control for RAM regions.*

Computer Architecture

End of Chapter 7 – Processor and Memory Architectures



VIVES University of Applied Sciences
Bachelor in electronics-ICT

